PROJECT REPORT

ON

IoT DAILY INTELLIGENCE SYSTEM

ESP32-Based Environmental Monitoring with Flask, Supabase and Gemini AI

Submitted By

Bhavya Chouhan

B.Tech CSE (IoT, Cyber Security and Blockchain Technology)

SIES Graduate School of Technology

Academic Year: 2026–2027

CERTIFICATE

This is to certify that the project entitled “IoT Daily Intelligence System” is a project work carried out as part of the academic requirements of the B.Tech CSE (IoT, Cyber Security and Blockchain Technology) program. The project implements an ESP32-based environmental monitoring system integrated with a Python Flask backend, Supabase database, web dashboard, and Gemini AI analysis.

DECLARATION

I hereby declare that the project report titled “IoT Daily Intelligence System” is my own work prepared for academic purposes. The implementation described in this report is based on the ESP32 sensor system, Flask backend, Supabase database, web dashboard, and Gemini AI integration developed for this project.

Name: Bhavya Chouhan
Signature: digitally signed by bhavya chouhan
Date: 4 October 2026

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my faculty members and institution and technical team for providing the opportunity and support to work on this project. I am thankful for the guidance and resources that helped me understand IoT sensors, ESP32 programming, Python Flask, database connectivity, web development, and AI integration. I also appreciate the open-source tools and platforms used during the development of this system.

ABSTRACT

The IoT Daily Intelligence System is an environmental monitoring project that collects real-time environmental data using an ESP32 DevKit V1 and multiple sensors. The system measures temperature and humidity using a DHT11 sensor, atmospheric pressure using a BMP180 sensor, gas-sensor response using an MQ135 module, and rain detection using a rain sensor. The ESP32 sends the sensor readings over Wi-Fi to a Python Flask backend. The backend stores the readings in a Supabase PostgreSQL database and provides APIs for retrieving recent sensor information.

A web dashboard displays current readings and historical charts using HTML, CSS, JavaScript, and Chart.js. Google Gemini is integrated through the Gemini API to generate a short environmental analysis from the latest sensor readings. The MQ135 value is treated as a raw ADC reading rather than a calibrated AQI value. The project demonstrates the integration of embedded systems, cloud databases, web technologies, and generative AI into a single IoT application.

TABLE OF CONTENTS

1. Introduction

2. Problem Statement

3. Objectives

4. Existing System

5. Proposed System

6. System Architecture

7. Hardware Requirements

8. Software Requirements

9. Hardware Components

10. Software Technologies

11. Circuit and Wiring Details

12. Working of the System

13. ESP32 Programming

14. Flask Backend

15. Supabase Database

16. Web Dashboard

17. Gemini AI Integration

18. Data Flow

19. Implementation

20. Results

21. Advantages

22. Limitations

23. Future Scope

24. Conclusion

25. References

1. INTRODUCTION

Internet of Things (IoT) systems connect physical devices and sensors to software platforms so that real-world information can be collected, stored, visualized, and analyzed. Environmental monitoring is an important IoT application because temperature, humidity, atmospheric pressure, rainfall, and gas-sensor readings can be collected continuously.

The IoT Daily Intelligence System combines an ESP32 microcontroller with environmental sensors, a Python Flask backend, a Supabase PostgreSQL database, a browser-based dashboard, and Gemini AI.

2. PROBLEM STATEMENT

Traditional sensor experiments often display values only through a serial monitor. This makes historical storage, browser-based visualization, and simple interpretation difficult. The project addresses this by collecting sensor readings, storing them in a database, visualizing them on a dashboard, and providing AI-assisted interpretation.

3. OBJECTIVES

• Collect environmental data using multiple sensors connected to ESP32.
• Send sensor readings over Wi-Fi to a Flask server.
• Store readings in a Supabase PostgreSQL database.
• Provide an API for retrieving recent readings.
• Display live readings and charts on a web dashboard.
• Integrate Gemini AI for simple environmental analysis.
• Keep raw measurements separate from AI interpretation.

4. EXISTING SYSTEM

A basic environmental monitoring setup commonly uses a microcontroller and serial monitor to display sensor values. Although useful for testing, it does not inherently provide centralized historical storage, browser visualization, or AI-assisted interpretation.

5. PROPOSED SYSTEM

The proposed system uses ESP32 as the sensing and communication device. Sensor readings are transmitted through Wi-Fi to a Python Flask server. Flask stores the readings in Supabase. A web dashboard retrieves recent records and displays them as cards and charts. Gemini AI receives the latest readings through Flask and generates an environmental summary.

6. SYSTEM ARCHITECTURE

ESP32 + Sensors → Wi-Fi → Flask Backend → Supabase Database → Web Dashboard.

For AI analysis: Supabase → Flask /ai-analysis → Gemini API → AI response → Web Dashboard.

7. HARDWARE REQUIREMENTS

- ESP32 DevKit V1
- DHT11 temperature and humidity sensor
- BMP180 pressure sensor
- MQ135 gas sensor
- Rain sensor module
- Jumper wires

• USB cable
• Computer running Ubuntu/Linux
• Wi-Fi hotspot/router

8. SOFTWARE REQUIREMENTS

• Arduino IDE
• ESP32 board support
• Python 3.14
• Flask
• python-dotenv
• Supabase Python client
• Google GenAI Python SDK
• HTML, CSS and JavaScript
• Chart.js
• Supabase PostgreSQL

9. HARDWARE COMPONENTS

ESP32 DevKit V1: main controller and Wi-Fi communication device.

DHT11: measures temperature and relative humidity.

BMP180: measures atmospheric pressure.

MQ135: provides a raw analog gas-sensor response through the ADC input. It is not directly treated as AQI.

Rain Sensor: provides rain detection signals; the digital signal is used for rain/no-rain status.

10. SOFTWARE TECHNOLOGIES

Python Flask provides the backend and API routes. Supabase provides cloud PostgreSQL storage. Gemini API provides natural-language analysis. HTML/CSS/JavaScript create the dashboard, and Chart.js provides sensor charts.

11. CIRCUIT AND WIRING DETAILS

DHT11 OUT → GPIO 4
BMP180 SDA → GPIO 21
BMP180 SCL → GPIO 22
Rain Sensor AO → GPIO 34
Rain Sensor DO → GPIO 33
MQ135 AO → GPIO 32
MQ135 DO → GPIO 2

VCC and GND are connected according to the sensor modules and ESP32 operating requirements.

12. WORKING OF THE SYSTEM

1. ESP32 initializes the sensors.
2. ESP32 connects to Wi-Fi.
3. Sensor readings are collected.
4. ESP32 creates a JSON payload.
5. JSON is sent to Flask /sensor.
6. Flask inserts readings into Supabase.
7. Dashboard requests recent readings through /dashboard.
8. Cards and charts are updated.
9. /ai-analysis retrieves the latest record and sends a controlled prompt to Gemini.
10. Gemini returns an environmental analysis for display.

13. ESP32 PROGRAMMING

The ESP32 firmware is written in Arduino C/C++. It uses Wi-Fi and HTTP functionality to connect to the Flask server. Sensor values are collected and sent approximately every five seconds. The JSON payload contains temperature, humidity, pressure, MQ135 raw value, and rain status.

14. FLASK BACKEND

The Flask backend contains routes for receiving sensor data, retrieving dashboard data, serving the dashboard page, and generating Gemini analysis.

POST /sensor — receives sensor readings.
GET /dashboard — returns recent readings.
GET /dashboard-ui — serves the dashboard.
GET /ai-analysis — generates AI analysis.

15. SUPABASE DATABASE

Sensor readings are stored in the PostgreSQL table sensor_readings. Fields include id, created_at, temperature, humidity, pressure, mq135, and rain. Row-level security policies were configured for the prototype so the application can insert and retrieve required sensor data.

16. WEB DASHBOARD

The dashboard displays current temperature, humidity, pressure, MQ135 raw reading, and rain status. Chart.js is used for recent temperature, humidity, and pressure trends. Sensor data refreshes periodically and Gemini analysis refreshes at a slower interval. A manual AI refresh button is also provided.

17. GEMINI AI INTEGRATION

Gemini is integrated using the Google GenAI Python SDK. Flask retrieves the latest sensor record and constructs a prompt containing the measured values. The AI is instructed not to invent measurements. The MQ135 reading is explicitly identified as a raw ADC value and is not presented as a calibrated AQI or pollutant concentration.

18. DATA FLOW

Sensors → ESP32 → Wi-Fi → Flask /sensor → Supabase sensor_readings → Flask /dashboard → Web Dashboard.

AI flow: Supabase → Flask /ai-analysis → Gemini API → AI response → Web Dashboard.

19. IMPLEMENTATION

The project was implemented incrementally. Sensors were tested with ESP32, Wi-Fi communication was established using a mobile hotspot, Flask was configured to receive JSON data, Supabase was configured for persistent storage, the dashboard was developed with HTML/CSS/JavaScript and Chart.js, and Gemini was integrated for environmental analysis.

20. RESULTS

The completed system successfully receives sensor data from ESP32 and stores it in Supabase. The dashboard displays current values and recent charts. Example observed readings included approximately: Temperature 32 °C, Humidity 47.4%, Pressure 1009.1 hPa, MQ135 approximately 1013 raw ADC, and no rain

detected. Gemini successfully generated analysis while treating MQ135 as an uncalibrated raw reading.

21. ADVANTAGES

• Real-time environmental monitoring
• Historical data storage
• Browser-based dashboard
• Cloud database integration
• AI-assisted interpretation
• Modular architecture
• Easy extension with additional sensors
• Combines IoT, cloud, web and AI technologies

22. LIMITATIONS

• MQ135 readings are raw and require calibration for accurate air-quality assessment.
• The prototype depends on Wi-Fi connectivity.
• Flask currently runs in the development environment.
• Gemini analysis depends on API availability and usage limits.
• Current AI analysis focuses mainly on the latest reading rather than full long-term trend analysis.

23. FUTURE SCOPE

• 24-hour and 7-day trend analysis
• Automated daily environmental reports
• Email or notification alerts
• Sensor anomaly detection
• Better MQ135 calibration and air-quality estimation
• Additional weather/environment APIs
• User authentication

• Mobile application
• Cloud deployment
• Scheduled AI reports
• Predictive analysis
• Additional sensors such as light, soil moisture, or particulate matter sensors

24. CONCLUSION

The IoT Daily Intelligence System demonstrates how embedded hardware, wireless communication, cloud databases, web development, and generative AI can be combined into one practical application. ESP32 collects environmental information and sends it to Flask. Supabase provides persistent storage, the web dashboard provides visualization, and Gemini provides natural-language interpretation. The project provides a strong foundation for future environmental intelligence features.

25. REFERENCES

1. Arduino IDE and ESP32 development resources.
2. Flask and Python web development documentation.
3. Supabase and PostgreSQL documentation.
4. Google Gemini API / Google GenAI SDK documentation.
5. Chart.js documentation.
6. DHT11, BMP180, MQ135 and rain sensor technical documentation.

APPENDIX A – PROJECT COMPONENT SUMMARY

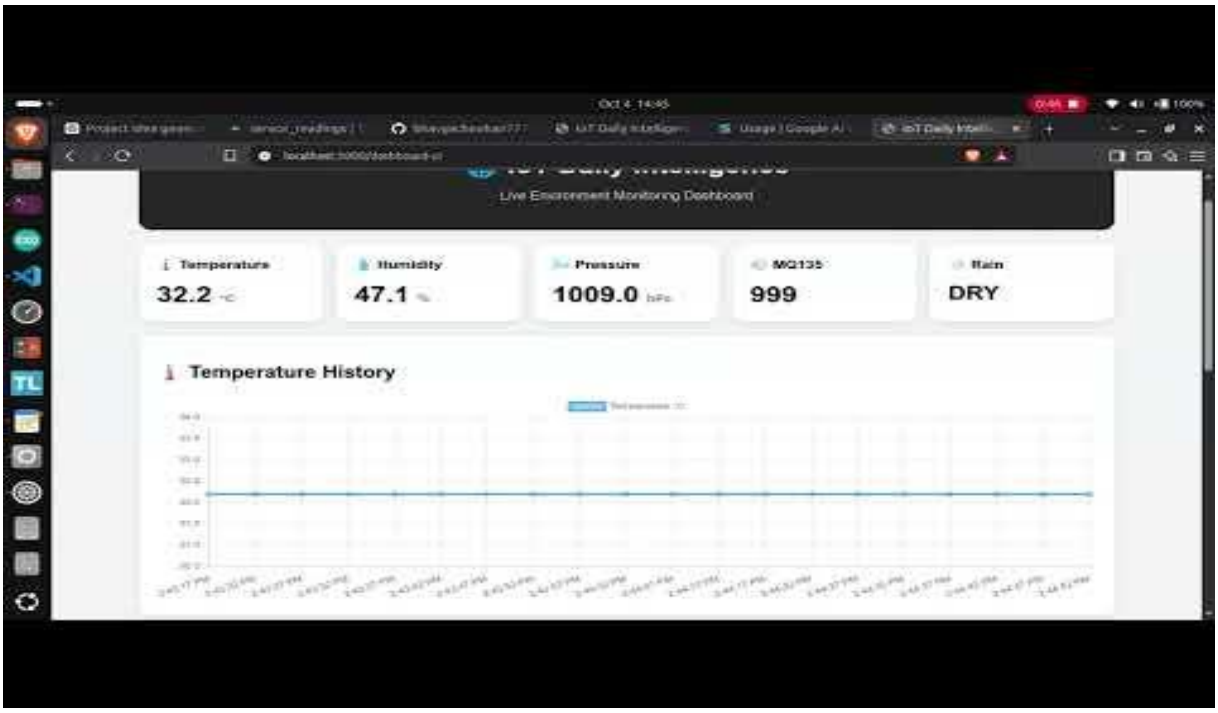
Component	Purpose	Connection / Technology
ESP32 DevKit V1	Main controller and Wi-Fi	Arduino C/C++
DHT11	Temperature and humidity	GPIO 4
BMP180	Atmospheric pressure	I2C: SDA 21, SCL 22
Rain Sensor	Rain detection	AO 34, DO 33
MQ135	Raw gas-sensor response	AO 32, DO 2
Flask	Backend/API	Python
Supabase	Cloud database	PostgreSQL
Gemini	AI analysis	Google GenAI API
Chart.js	Dashboard charts	JavaScript

APPENDIX B – SCREENSHOTS TO INSERT

- ESP32 Serial Monitor showing Wi-Fi connection and sensor readings.
- Flask terminal showing successful POST /sensor requests.
- Supabase sensor_readings table.
- Supabase query showing recent readings.
- Final web dashboard with live sensor cards and charts.
- Gemini /ai-analysis response.
- Final dashboard with AI Environmental Intelligence section.

ACTUAL GALLERY OF MY PROJECT– SCREENSHOTS, SCREENCASTS AND FOOTAGE

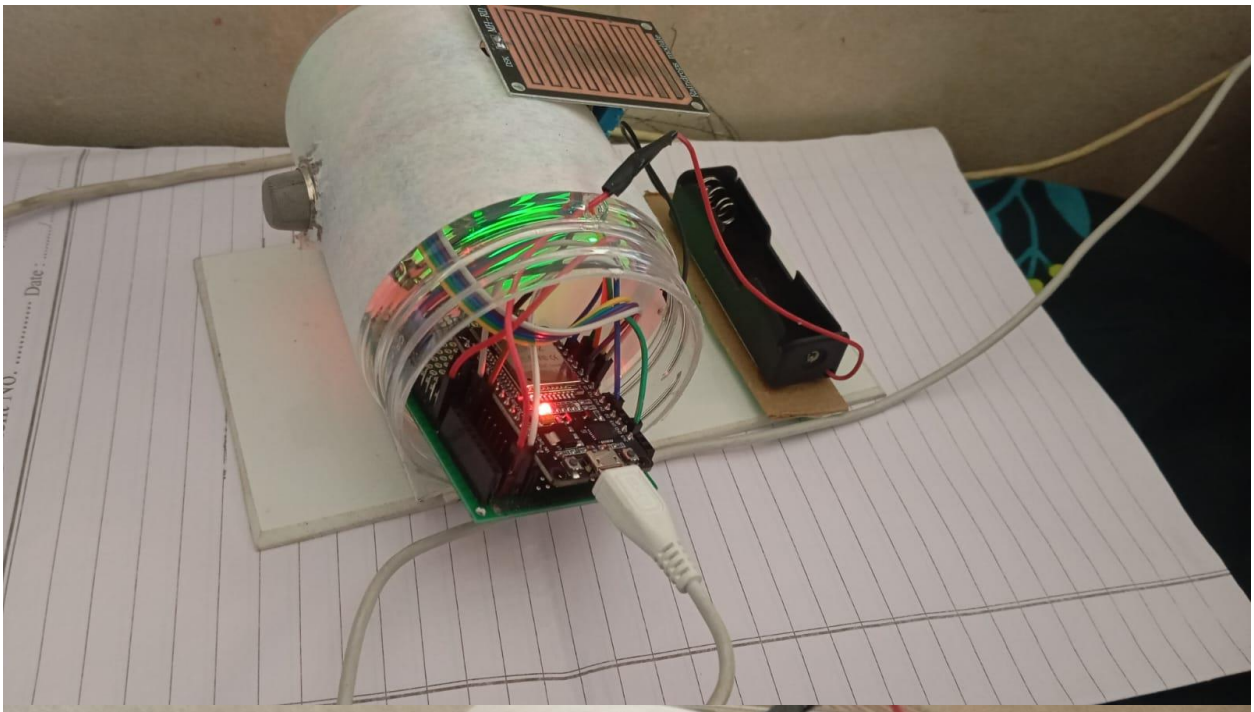
DASHBOARD UI FOOTAGE:

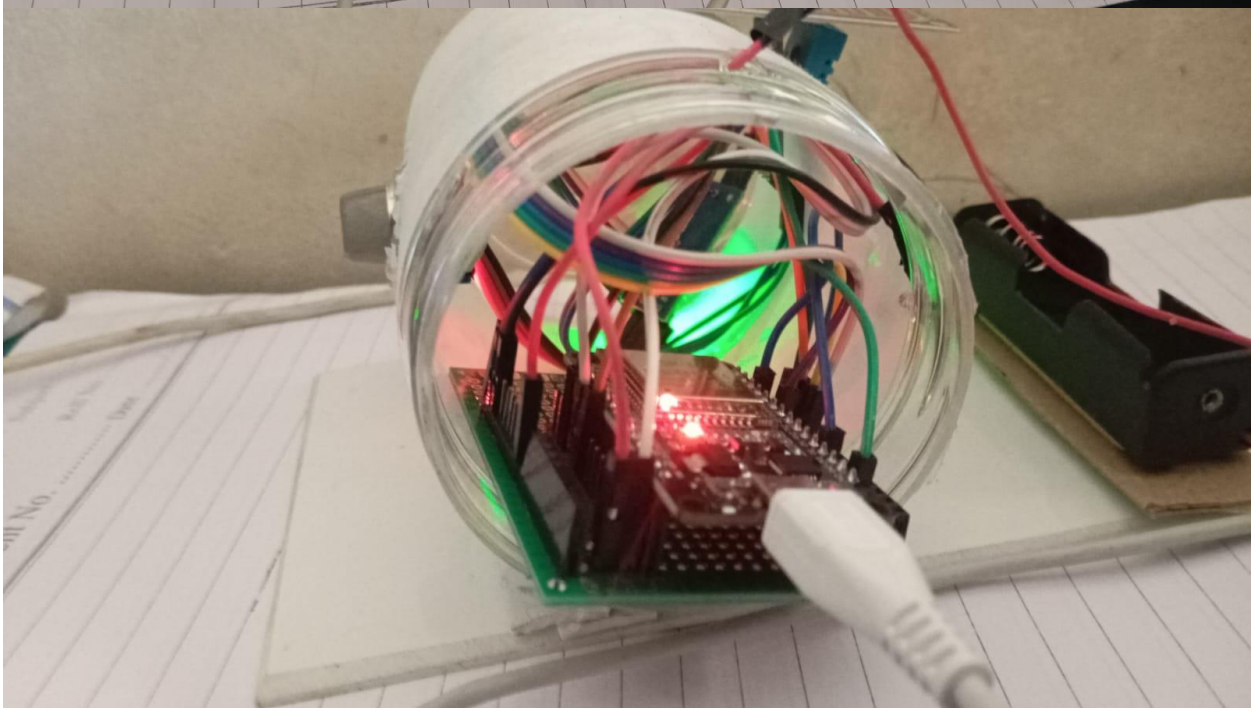


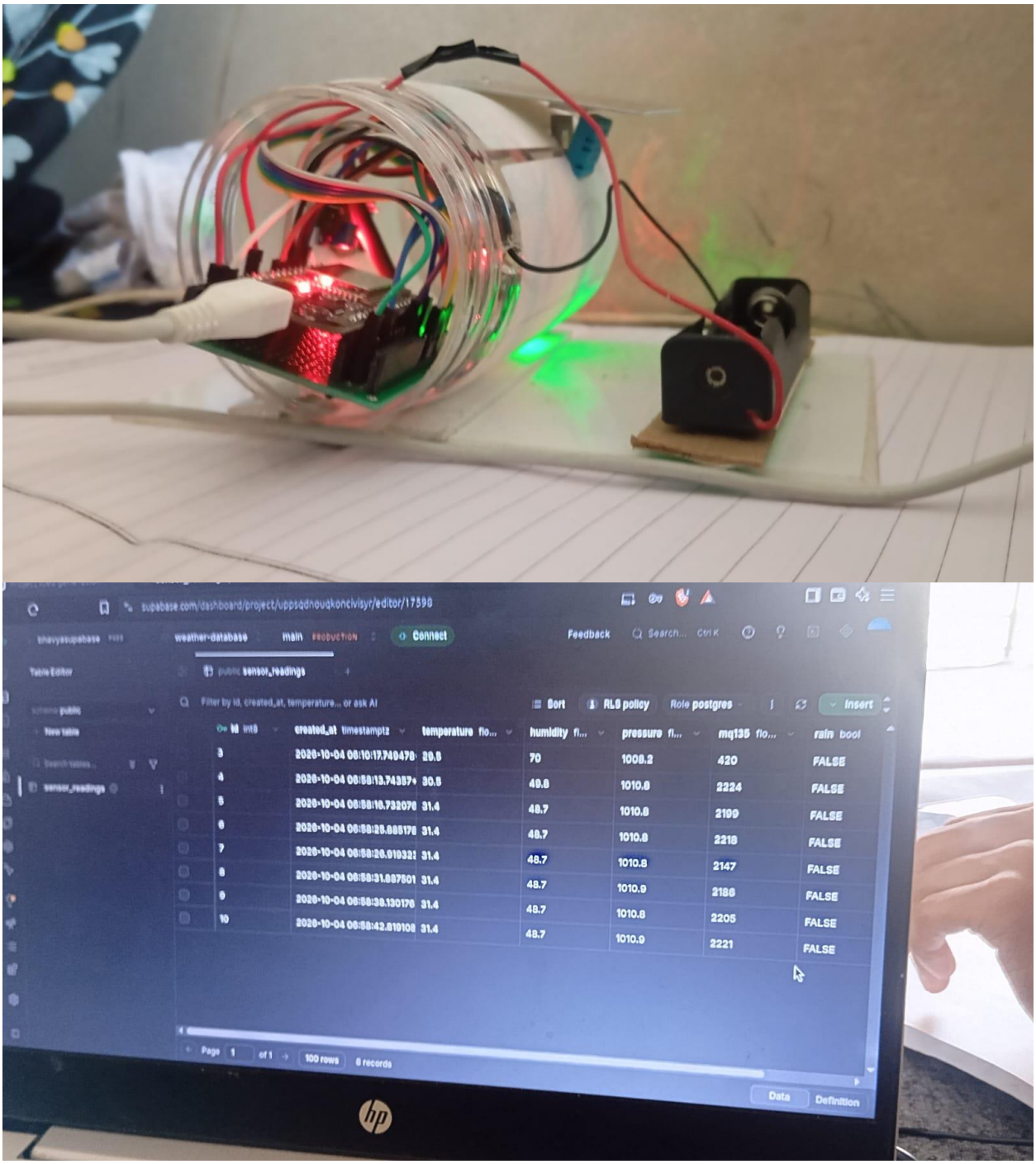
https://youtu.be/62JcSHq21Sc

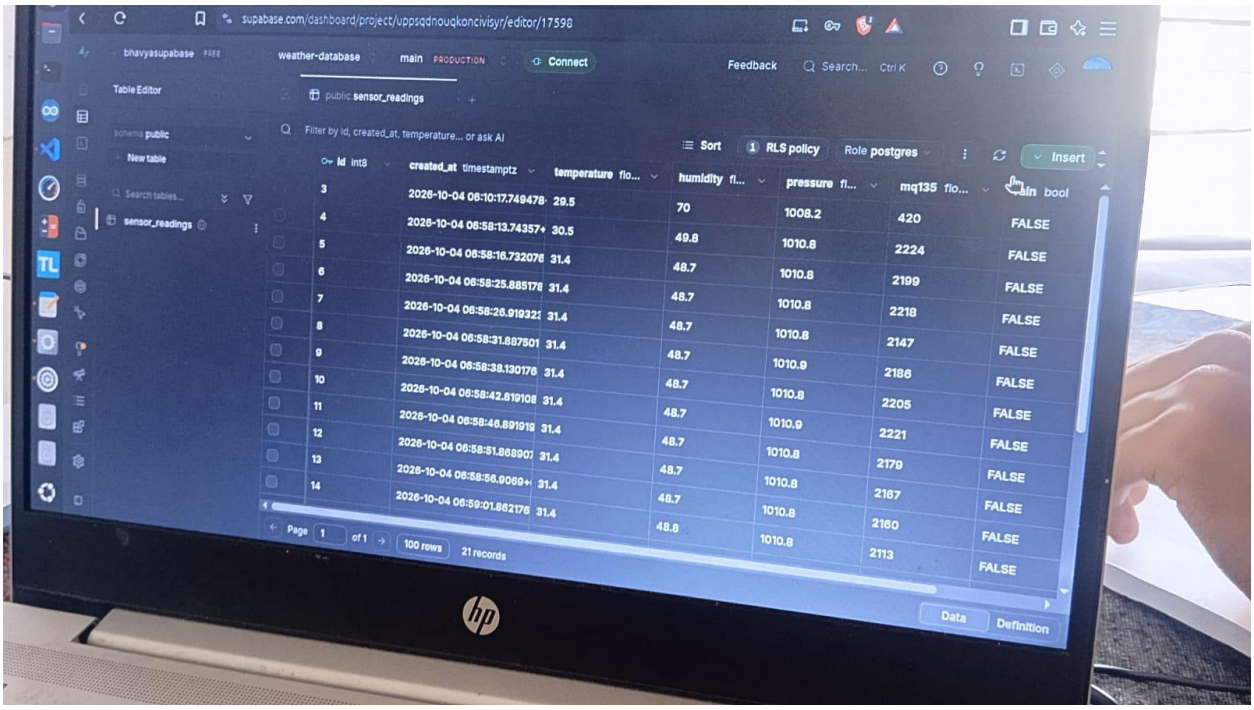
HARDWARE WORKING FOOTAGE:

https://youtu.be/5ALvSxDlkFE






supabase.com/dashboard/project/uppsqdnoudkoncivisyr/editor/17596
weather-database
main
Connect
Feedback
Table Editor
public sensor_readings
Filter by id, created_at, temperature... or ask AI
Sort
RLS policy
Role postgres
Insert
id int8
created_at timestamptz
temperature flo...
humidity fl...
pressure fl...
mq135 flo...
rain bool
3 2020-10-04 08:10:17.740478 29.8 70 1008.2 420 FALSE
4 2020-10-04 08:58:13.743874 30.5 49.8 1010.8 2224 FALSE
5 2020-10-04 08:58:19.732078 31.4 48.7 1010.8 2199 FALSE
6 2020-10-04 08:58:25.885178 31.4 48.7 1010.8 2218 FALSE
7 2020-10-04 08:58:26.919321 31.4 48.7 1010.8 2147 FALSE
8 2020-10-04 08:58:31.887501 31.4 48.7 1010.9 2186 FALSE
9 2020-10-04 08:58:36.130178 31.4 48.7 1010.8 2205 FALSE
10 2020-10-04 08:58:42.819108 31.4 48.7 1010.9 2221 FALSE
Page 1 of 1
100 rows
8 records
Data
Definition



My GITHUB REPO LINK:

https://github.com/bhavyachouhan777/weather-repo.git